ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50) Nature

of Assignment: Individual

Duration: 1 Week

Submission Date: 10/11/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level) Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

Name : THANISHKA S

Reg no : 192521457

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time
complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least
Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects

Bloom's Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

- The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each
- How the same problem would be written in a procedural style, and what changes in an object-oriented style
- The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism

– and where each could appear in this system

- The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)

You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

1. Aim

To analyse the metro smart-card fare system using Object-Oriented Programming concepts and identify its objects, attributes, behaviours and rules.

2. Problem Description

The Namma Metro system has three zones:

- Inner Zone – Rs. 10
- Middle Zone – Rs. 20
- Outer Zone – Rs. 30

Additional conditions:

- A 20% surcharge is applied during peak hours (8–10 AM and 5–8 PM).
- The passenger must maintain a minimum balance of Rs. 50 after fare deduction.
- Students and senior citizens receive concessions.
- The system must calculate fares accurately without negative balances.

3. Identify the Objects

Object	**Data (Attributes)**	**Behaviour (Methods)**
SmartCard	Card ID, Balance	Check balance, Deduct fare, Recharge
Passenger	Passenger ID, Name, Category	Check passenger type, Apply concession
Journey	Source, Destination, Time	Calculate journey fare
Zone	Zone name, Zone fare	Identify travel zone
FareTable	Base fare, Surcharge	Calculate final fare

4. Procedural Programming vs Object-Oriented Programming Procedural Programming:

- Uses separate variables and functions.
- All fare calculations are handled through functions.
- Data is not properly protected.
- Changes in fare rules may require modifying multiple functions.

Object-Oriented Programming:

- Uses classes and objects.
- Combines data and functions into a single unit.
- Protects data using private access modifiers.
- Makes the system easier to maintain and modify.

5. Basic Principles of OOP

1. **Encapsulation**
   - Combines data and functions inside a class.
   - Keeps the smart-card balance private.
   - Prevents unauthorized balance modification.
2. **Abstraction**
   - Hides unnecessary implementation details.
   - Passengers only need to use the card without knowing the internal fare calculation.
3. **Inheritance**
   - Creates different passenger categories using a common Passenger class.
   - Student and SeniorCitizen classes can inherit common passenger properties. **4. Polymorphism**
   - Allows different passenger categories to calculate concessions differently.
   - The same fare calculation method can produce different results based on passenger type.

6. Rules for a Correct Journey

1. The passenger must have a valid smart card.
2. The journey must have a valid destination zone.
3. Peak-hour surcharge must be calculated correctly.

4. The fare must be deducted accurately.
5. The remaining balance must be at least Rs. 50.
6. The balance must never become negative.

7. Alternative Concept and Complexity Analysis

If the system uses procedural programming instead of OOP, separate variables and functions are used.

Example:

```
#include <iostream>

using namespace std; int

balance = 100; void

deductFare(int fare)

{     if (balance - fare >= 50) {

balance = balance - fare;         cout <<

"Journey Allowed" << endl;

    }

else

    {

        cout << "Journey Denied" << endl;

    }

} int main() {

deductFare(20);

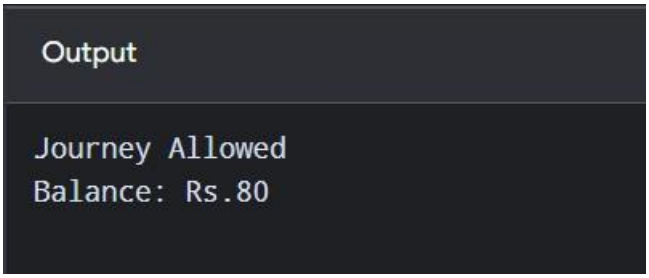
cout << "Balance:
```

Rs." << balance;

return 0;

}

Output:



8. Complexity Analysis

Operation	**Procedural**	**OOP**
Fare calculation	O(1)	O(1)
Balance checking	O(1)	O(1)
Memory per card	O(1)	O(1)
Maintenance	Less flexible	More flexible

Explanation:

Both approaches require constant time for fare calculation.

Both use constant memory per card.

OOP is easier to maintain because data and functions are organized into classes.

If a fare rule changes, OOP generally requires fewer modifications.

9. Result

Thus, the metro smart-card fare system was analysed using Object-Oriented Programming concepts. The important objects, their attributes, behaviours and journey rules were identified. OOP provides better data security, reusability and maintainability compared with procedural

programming.

TASK 2 Use Tokens, Keywords, Identifiers and Constants Correctly

Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one place only. **Discuss:**

- The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples
- #define, const and constexpr constants, and why constexpr is preferred for the fare table
- enum and enum class for zones, and why enum class is safer
- Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

1. Aim

To understand and use C++ tokens, keywords, identifiers and constants to define metro fare values and limits correctly.

2. Five Types of Tokens in C++

Tokens are the smallest individual elements in a C++ program.

Token Type	Description	Examples
Keywords	Reserved words in C++	int, class, if, return
Identifiers	Names given to variables and functions	balance, fare, SmartCard
Constants	Fixed values that cannot be changed	1000, 2000, 3000

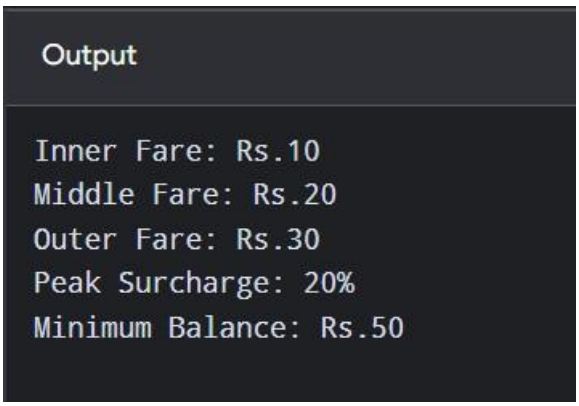
Operators	Symbols used to perform operations	+, -, *, /, %
Punctuators	Symbols used to structure programs	;, {}, (), []

3. Define Fares Using Constants #include <iostream> using namespace std; constexpr int INNER_FARE = 1000; constexpr int MIDDLE_FARE = 2000; constexpr int OUTER_FARE = 3000; constexpr int PEAK_SURCHARGE = 20; constexpr int MIN_BALANCE = 5000; int main() {

cout << "Inner Fare: Rs." << INNER_FARE / 100 << endl; cout << "Middle Fare: Rs." << MIDDLE_FARE / 100 << endl; cout << "Outer Fare: Rs." << OUTER_FARE / 100 << endl; cout << "Peak Surcharge: " << PEAK_SURCHARGE << "%" << endl; cout << "Minimum Balance: Rs." << MIN_BALANCE / 100; return 0;

}

4. Output



5. Difference Between #define, const and constexpr

Type	**Description**	**Example**
#define	Preprocessor macro	#define FARE 1000
const	Creates a read-only variable	const int FARE = 1000;
constexpr	Defines a compile-time constant	constexpr int FARE = 1000;

Why constexpr is preferred:

Values are evaluated at compile time.

Provides better type checking.

Makes the program easier to maintain.

Avoids repeated fare values.

Suitable for fixed metro fare calculations.

6. enum and enum class enum:

Defines a group of named integer constants. Values can sometimes conflict with other identifiers. **enum class:**

Provides better type safety.

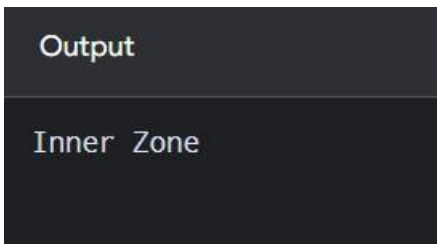
Keeps values inside their own scope.

Prevents accidental conversions to integers.

Example:

```
#include <iostream>
using namespace
std; enum class Zone
{
   Inner,
   Middle,
   Outer
}; int
main()
{
   Zone z = Zone::Inner;
if (z == Zone::Inner)
cout << "Inner Zone";
return 0;
}
```

Output:



7. Naming Rules and Conventions

Identifiers must begin with a letter or underscore.

Spaces are not allowed in identifiers.

C++ keywords cannot be used as identifiers.

Variable names should be meaningful.

Constants should preferably use uppercase letters.

Examples:

balance – variable.

calculateFare() – function.

SmartCard – class.

MIN_BALANCE – constant.

8. Alternative Concept and Complexity Analysis

If #define macros are used instead of constexpr and enum class, the program becomes less typesafe.

Example:

#define INNER_FARE 1000

#define MIDDLE_FARE 2000

#define OUTER_FARE 3000

Comparison	**#define**	**constexpr**
Type checking	No	Yes
Scope	Preprocessor-wide	Normal C++ scope
Memory	Usually no storage	Usually no storage
Execution time	No runtime overhead	No runtime overhead
Safety	Less safe	More safe

Time Complexity: O(1)

Space Complexity: O(1) for the fixed fare values.

Explanation:

Both approaches have almost the same execution performance. However, constexpr provides better type checking, readability and maintainability.

9. Result

Thus, C++ tokens, keywords, identifiers and constants were studied and implemented. The constexpr keyword and enum class are preferred because they provide better type safety, readability and maintainability for the metro smart-card fare system.

TASK 3 Choose Data Types and Check Type Compatibility

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

- The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system
- Why money should be stored in integer paise and not as double rupees
- Type compatibility rules, and what the compiler does with int, double and bool values in the same expression
- The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

1. Aim

To select suitable C++ data types for the metro smart-card fare system and understand type compatibility, memory size and precision.

2. Data Types Used in the System

Data	**Data Type**	**Reason**
Card ID	int	Stores card number
Balance	long long	Stores money in paise

Fare	long long	Stores fare accurately
Passenger name	string	Stores passenger name
Peak hour	bool	Stores true or false
Zone number	int	Stores zone value
Journey count	long long	Stores large journey counts
Time	int	Stores hour value
Concession	double	Stores percentage if needed

3. Why Money Should Be Stored in Integer Paise Money should be stored as integer paise instead of double rupees.

Example:

Rs. 10 = 1000 paise

Rs. 20 = 2000 paise

Rs. 30 = 3000 paise **Advantages:**

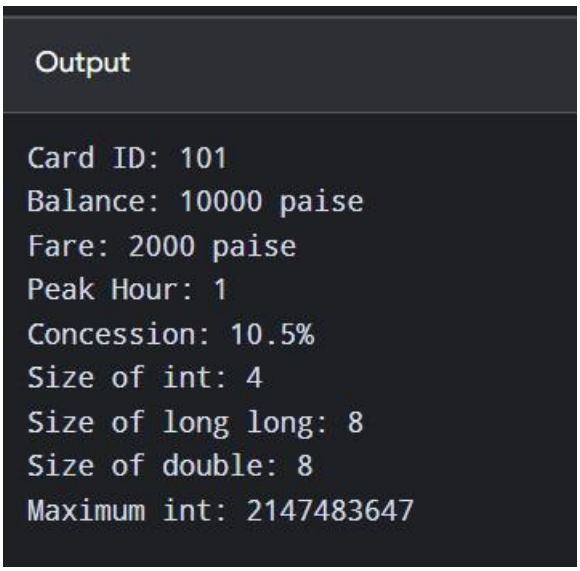
1. Avoids floating-point rounding errors.
2. Provides accurate fare calculations.
3. Prevents paisa differences.
4. Makes balance calculations reliable.

4. C++ Program to Demonstrate Data Types

#include <iostream> #include <climits> using namespace std; int main() { int cardID = 101; long long balance = 10000; long long fare = 2000; bool peak = true;

```
double concession = 10.5;     cout << "Card ID: " << cardID
<< endl;     cout << "Balance: " << balance << " paise" <<
endl;     cout << "Fare: " << fare << " paise" << endl;     cout
<< "Peak Hour: " << peak << endl;     cout << "Concession:
" << concession << "%" << endl;     cout << "Size of int: "
<< sizeof(int) << endl;     cout << "Size of long long: " <<
sizeof(long long) << endl;     cout << "Size of double: " <<
sizeof(double) << endl;     cout << "Maximum int: " <<
INT_MAX << endl;
    return 0;
}
```

5. Output



6. Type Compatibility

Type compatibility means converting one data type into another during an operation.

Example:

int a = 10;

double b = 5.5;

double c = a + b;

Here:

int is converted into double.

The addition produces a double result.

The result is 15.5.

7. Why int May Not Be Enough

Assume each journey generates Rs. 30.

Journeys	**Total Collection**
20,000	Rs. 6,00,000
5,00,000	Rs. 1,50,00,000
30,00,000	Rs. 9,00,00,000

In paise:

30,00,000 × 3000 = 9,000,000,000 paise.

This exceeds the maximum value of a typical 32-bit int (2,147,483,647).

Therefore, long long is recommended.

8. Alternative Concept and Complexity Analysis

If money is stored using double instead of long long, the fare calculation would be performed using decimal values.

Example:

double balance = 100.00;

double fare = 20.00;

balance = balance - fare;

cout << balance; **Output:**

80

Comparison	**long long**	**double**
Storage	8 bytes	8 bytes
Precision	Exact integer paise	Approximate decimal
Addition	O(1)	O(1)
Memory per value	O(1)	O(1)
Accuracy	High for money	Rounding errors possible

Explanation:

Both approaches take constant time for addition.

Both use constant space per value.

long long provides exact integer arithmetic for paise.

double may introduce rounding errors when decimal values are added repeatedly.

9. Result

Thus, suitable C++ data types were selected for the metro smart-card fare system. Integer paise using long long is recommended for accurate money calculations and to prevent overflow when processing millions of journeys.

TASK 4 Control Implicit Conversions

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

- Integer division and integer-to-double conversion
- Comparison of signed and unsigned values, and the compiler warning it produces

• Narrowing conversions, and how brace initialisation (int x{3.7};) prevents them • static_cast and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
```

```
{   int a = 7, b =

2;

 double c = a / b;
 double d = static_cast<double>(a) / b;   std::cout << c

<< ' ' << d << '\n';  char ch = 'A' + 2;   std::cout << ch

<< ' ' << ch + 1 << '\n';   bool flag = 5;   std::cout <<

flag << ' ' << std::boolalpha << flag << '\n';   unsigned

int u = 1;


 int s = -1;   std::cout << (s < u ? "s < u" : "s

>= u") << '\n';

}
```

My predicted output (write before running): Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

1. Aim

To understand implicit type conversions in C++ and use explicit type conversions to avoid unexpected results in the metro smart-card fare system.

2. Implicit Type Conversion

Implicit conversion happens when the compiler automatically converts one data type into another.

Types of conversions:

1. Integer division.
2. Integer-to-double conversion.
3. Signed-to-unsigned conversion.
4. Integer-to-boolean conversion.
5. Character-to-integer conversion.

3. Explanation of Type Conversions

1. Integer Division

When two integers are divided, the decimal part is removed.

Example:

int a = 7, b = 2;

double c = a / b;

cout << c; Output:

3

2. Integer-to-Double Conversion

An integer can be converted into a double using static_cast.

double d = static_cast<double>(a) / b; cout << d; Output:

3.5

3. Character-to-Integer Promotion

A character is automatically converted into its ASCII value during arithmetic operations.

4. Integer-to-Boolean Conversion

0 becomes false.

Any non-zero value becomes true.

5. Signed-to-Unsigned Conversion

A negative signed integer converted to unsigned becomes a large positive value.

4. Predict the Output Program:

```
#include <iostream> using namespace std; int

main() {     int a = 7, b = 2;     double c = a / b;

double d = static_cast<double>(a) / b;     cout <<

c << ' ' << d << '\n';     char ch = 'A' + 2;     cout

<< ch << ' ' << ch + 1 << '\n';     bool flag = 5;

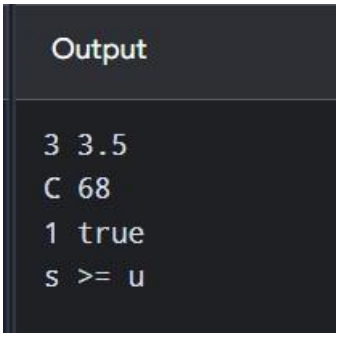
cout << flag << ' ' << boolalpha << flag << '\n';

unsigned int u = 1;     int s = -1;     cout << (s < u

? "s < u" : "s >= u") << '\n';     return 0;

}
```

5. Predicted Output



6. Explanation of Actual Output

Statement	Explanation
a / b	Integer division gives 3.
static_cast<double>(a)/b	Converts 7 into double and gives 3.5.
'A' + 2	Character arithmetic produces C.
ch + 1	Character C is promoted to integer 67 and then 1 is added, giving 68.
bool flag = 5	Non-zero integer becomes true.
boolalpha	Displays true instead of 1.
s < u	Signed integer is converted to unsigned, so the comparison is false.

7. Corrected Code Using Explicit Conversion #include <iostream> using

namespace std; int main() { int a{7}, b{2}; double c =

static_cast<double>(a) / b;

cout << c << endl; int value =

static_cast<int>('A') + 2; char ch =

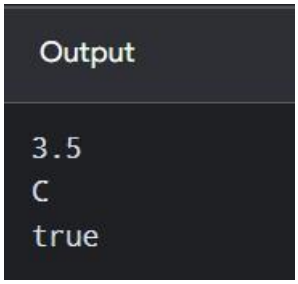
static_cast<char>(value); cout <<

ch << endl; bool flag = (5 != 0);

cout << boolalpha << flag << endl;

return 0; }

8. Output



9. Alternative Concept and Complexity Analysis

If all conversions are written explicitly using static_cast and brace initialization, the program becomes easier to understand and safer.

Comparison	**Implicit Conversion**	**Explicit Conversion**
Conversion	Automatic	Programmer controlled
Readability	Less clear	More clear
Type safety	Less control	Better control
Execution time	O(1)	O(1)
Memory	O(1)	O(1)

Advantages:

1. Prevents unexpected conversion errors.
2. Improves code readability.

3. Helps identify narrowing conversions.
4. Makes the program easier to debug.

10. Result

Thus, implicit and explicit type conversions in C++ were studied and implemented. Using static_cast and brace initialization helps prevent unexpected conversion errors and improves the reliability of the metro smart-card fare system.

TASK 5 Write Correct Expressions with the Right Precedence

Bloom's Level: K5 – Evaluate

Write the expressions for the peak check, the surcharge and the concession.

Discuss:

• Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment

• The difference between x++ and ++x, and why i = i++ is a mistake

• The result of / and % with negative operands

• Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()

{   int x = 5, y = 3;   std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';   int z

= x++ + ++y;   std::cout << z << ' ' << x << ' ' << y << '\n';   std::cout << (x

> y ? x : y) << ' ' << x % 4 * 2 << '\n';   std::cout << 10 / 3 << ' ' << 10 % 3

<< ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';   int k = 0;   for (int i = 0; i < 10; i

+= 3) k += i;


 std::cout << k << '\n';

}
```

My predicted output (write before running):
For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

1. Aim

To understand operator precedence and associativity in C++ and write correct expressions for metro fare calculation, peak-hour surcharge and passenger concessions.

2. Expressions Used in the Metro Fare System

1. Peak-Hour Check

(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)

Peak hours are 8 AM to before 10 AM.

Evening peak hours are 5 PM to before 8 PM.

Logical AND (&&) checks each time range.

Logical OR (||) combines both conditions.

2. Surcharge Calculation fare

= fare + (fare * 20 / 100);

Calculates a 20% surcharge.

Multiplication and division are performed before addition.

3. Concession Calculation fare

= fare - (fare * 10 / 100);

Calculates a 10%

concession.

Reduces the original fare.

3. Operator Precedence and Associativity

Operator	**Description**	**Precedence**
()	Parentheses	Highest
* / %	Multiplication, division, modulus	High
+ -	Addition, subtraction	Medium
< > >= <=	Relational operators	Lower
== !=	Equality operators	Lower
&&	Logical AND	Low
`		`
?:	Conditional operator	Very low
=	Assignment	Very low

4. Difference Between x++ and ++x Post-increment (x++):

Uses the current value first.

Increases the value afterwards.

Pre-increment (++x):

Increases the value first.

Uses the updated value.

Example:

int x = 5; cout <<

x++ << endl; cout <<

++x << endl;

Output:

5

7

Why i = i++ is a mistake:

It modifies and assigns the same variable in an unnecessarily confusing way. Avoid this expression and use i++ or ++i separately. **5. Division and Modulus with Negative Numbers** Example:

cout << 10 / 3 << endl; cout

<< 10 % 3 << endl; cout <<

-10 / 3 << endl; cout << -10

% 3 << endl; **Output:**

3

1

-3

-1

C++ integer division truncates towards zero, and the remainder has the sign of the dividend.

6. Predict the Output Program:

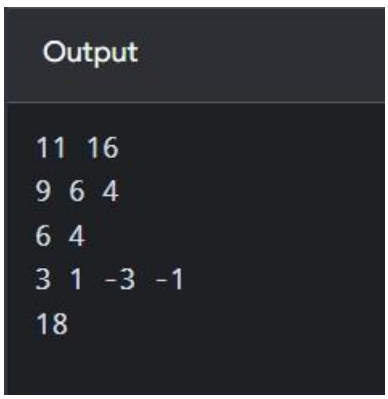
#include <iostream> using namespace std; int

main() { int x = 5, y = 3; cout << x + y * 2 <<

' ' << (x + y) * 2 << '\n'; int z = x++ + ++y;

```
cout << z << ' ' << x << ' ' << y << '\n';     cout <<
(x > y ? x : y) << ' ' << x % 4 * 2 << '\n';     cout <<
10 / 3 << ' ' << 10 % 3 << ' '          << -10 / 3 << ' '
<< -10 % 3 << '\n';     int k = 0;     for (int i = 0; i <
10; i += 3)         k += i;     cout << k << '\n';
return 0;
}
```

7. My Predicted Output



8. Explanation of Each Output Line 1:

y * 2 = 3 * 2 = 6

x + 6 = 11

(x + y) * 2 = 16 **Line**

2:

x++ uses 5 and then increases x to 6.

++y increases y to 4.

z = 5 + 4 = 9.

Line 3:

x > y means 6 > 4.

Conditional operator selects 6.

x % 4 * 2 = 2 * 2 = 4.

Line 4:

Integer division gives 3 and -3.

Modulus gives 1 and -1.

Line 5:

Loop values: 0, 3, 6, 9.

Total = 0 + 3 + 6 + 9 = 18.

9. Alternative Concept and Complexity Analysis

If a lookup table is used instead of logical AND and OR expressions, peak hours can be checked using an array.

Example:

```
#include <iostream> using namespace std; int

main() {     bool peak[24] = {};     for (int i =

8; i < 10; i++)         peak[i] = true;     for (int i

= 17; i < 20; i++)         peak[i] = true;     int

hour = 9;     if (hour >= 0 && hour < 24 &&

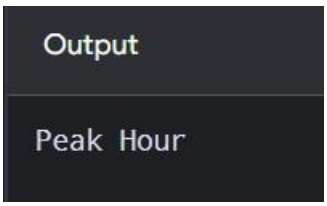
peak[hour])         cout << "Peak Hour";

   else         cout << "Off-Peak

Hour";     return 0;

}
```

Output:



Complexity Analysis:

Operation	**Logical Expression**	**Lookup Table**
Time complexity	O(1)	O(1)
Space complexity	O(1)	O(1)
Comparisons	Multiple	Fewer per check
Maintenance	Modify conditions	Modify table

The lookup table requires 24 Boolean entries but makes peak-hour changes easier to manage.

10. Result

Thus, operator precedence, associativity, increment operators, division and modulus operations were studied and implemented. Correct expressions help calculate metro fares, peak-hour surcharges and passenger concessions accurately.

TASK 6 Design the Control Structures

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's journeys.
Discuss:

- When to use if–else, a switch on the zone, and the conditional operator
- How a for loop, a range-based for loop and a while loop could process the journeys, and which is best
- Where break and continue are needed (for example, skipping an invalid journey)
- What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>

#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30 constexpr int
PEAK_SURCHARGE_PERCENT = 20;  constexpr long long MIN_BALANCE_PAISE
= 5000; // Rs 50 must remain on the card  long long fare(Zone z, bool peak)

{

 // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE,
then add  // PEAK_SURCHARGE_PERCENT % in peak hours using
integer arithmetic only
}

bool canTravel(long long balance, long long f)
```

```
{
 // TODO 2: Return true only if the balance left after paying the fare is at least
// MIN_BALANCE_PAISE
}
int main()  {   long long balance = 10000; // Rs
100.00 in paise   struct Trip { Zone zone; bool
peak; };
 Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true},
{Zone::Middle, false}};  for (const Trip& t : trips) {
long long f = fare(t.zone, t.peak);   if
(canTravel(balance, f)) {   balance -= f;   std::cout <<
"Fare Rs " << f / 100 << '.' << std::setw(2)
 << std::setfill('0') << f % 100 << std::setfill(' ');
 } else {
 std::cout << "Denied (low balance)";
 }   std::cout << " balance Rs " << balance / 100 <<
'\n';
 }
}
```

Expected output:

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

1. Aim

To design and implement C++ control structures for calculating metro fares, checking peak hours and processing passenger journeys.

2. Control Structures Used

1. if else Statement

Used to check whether the passenger has sufficient balance.

Allows or denies the journey based on the remaining balance.

2. switch Statement

Used to select the base fare according to the travel zone.

Inner – Rs. 10

Middle – Rs. 20

Outer – Rs. 30

3. Conditional Operator

Used to select a value based on a condition.

Example:

fare = peak ? fare + fare * 20 / 100 : fare;

3. Looping Statements

Loop	**Description**	**Usage**
for loop	Repeats a fixed number of times	Processing a fixed number of journeys
while loop	Repeats while a condition is true	Processing journeys until the user exits
Range-based for	Iterates through an array	Processing journey records

Best Choice: Range-based for loop is suitable for processing a fixed array of journeys because it is simple and readable.

4. Use of break and continue break – Terminates a loop or switch statement. continue – Skips the current iteration and moves to the next journey. default – Handles an invalid zone when no switch case matches.

5. Algorithmic Pseudocode

Algorithm: Metro Fare Calculation

START

DECLARE balance = 10000

READ zone

READ peak status

SWITCH zone

```
    CASE Inner:
fare = 1000
        BREAK
CASE Middle:
        fare = 2000
        BREAK
CASE Outer:
fare = 3000
        BREAK
    DEFAULT:
        PRINT "Invalid Zone"
        STOP
END SWITCH
IF peak is TRUE
    fare = fare + (fare * 20 / 100)
END IF
IF balance - fare >= 5000
balance = balance - fare
    PRINT "Journey Allowed"
    PRINT fare and balance
ELSE
    PRINT "Journey Denied"
END IF
```

STOP **6. Complete**

the Code C++

Program:

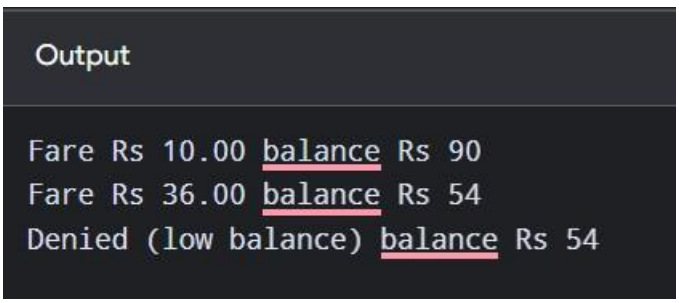
```
#include <iomanip> #include <iostream> using
namespace std; enum class Zone { Inner, Middle,
Outer }; constexpr int BASE_PAISE[] = {1000, 2000,
3000}; constexpr int
PEAK_SURCHARGE_PERCENT = 20; constexpr
long long MIN_BALANCE_PAISE = 5000; long long
fare(Zone z, bool peak)
{     long long amount =
0;     switch (z)
    {         case
Zone::Inner:
            amount = BASE_PAISE[0];
            break;         case
Zone::Middle:
            amount = BASE_PAISE[1];
            break;
        case Zone::Outer:
            amount = BASE_PAISE[2];
            break;
default:
```

```
            return -1;
    }     if
(peak)
    {
        amount = amount + (amount * PEAK_SURCHARGE_PERCENT / 100);
    }
    return amount;
} bool canTravel(long long balance, long long
f)
{     return f >= 0 && balance - f >=
MIN_BALANCE_PAISE;
} int main() {     long long
balance = 10000;     struct
Trip
    {
        Zone zone;
bool peak;     };

    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
```

```
    };     for (const Trip& t :
trips)
    {         long long f = fare(t.zone,
t.peak);         if (canTravel(balance, f))
        {             balance -= f;             cout
<< "Fare Rs " << f / 100 << '.'
                << setw(2) << setfill('0') << f % 100
                << setfill(' ');
        }         else{             cout <<
"Denied (low balance)";
        }
        cout << " balance Rs " << balance / 100 << '\n';
    }
return 0;
}
```

7. Expected Output



8. Peak-Hour Boundary Testing Peak hours:

8:00 AM – Peak

10:00 AM – Off-Peak

5:00 PM – Peak

8:00 PM – Off-Peak

Correct expression:

(hour >= 8 && hour < 10) ||

(hour >= 17 && hour < 20)

9. Alternative Concept and Complexity Analysis

If an array is used instead of a switch statement, the base fare can be selected using the zone index.

Example: constexpr long long FARE[] = {1000,
2000, 3000}; long long amount = FARE[zone];

Comparison	**Switch**	**Array**
Time complexity	O(1)	O(1)
Space complexity	O(1)	O(1) additional space for lookup
Readability	Good	Simple
Fourth zone	Add another case	Add another array element

Explanation:

Both methods provide constant-time fare selection.

An array makes fare updates easier when new zones are introduced.

A switch statement provides clear handling of individual cases.

10. Result

Thus, C++ control structures such as if–else, switch, conditional operators and loops were implemented successfully. The program calculates metro fares using integer paise, applies peak-hour surcharges and checks the minimum balance before allowing a journey.

TASK 7 From Procedural to Object-Oriented

Bloom's Level: K5 – Evaluate Rewrite

the fare logic in two styles.

Discuss:

- A procedural version with separate variables and free functions
- An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()
- How the class prevents a negative balance
- Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

1. Aim

To implement the metro smart-card fare system using procedural and object-oriented programming and compare their advantages.

2. Procedural Programming

Procedural programming uses separate variables and functions to perform fare calculations.

C++ Program:

```
#include <iostream>

using namespace std; long

long balance = 10000;

bool canTravel(long long

fare) {     return balance -

fare >= 5000;

} void deductFare(long long

fare)

{     if

(canTravel(fare))

    {         balance -= fare;         cout <<

"Journey Allowed" << endl;

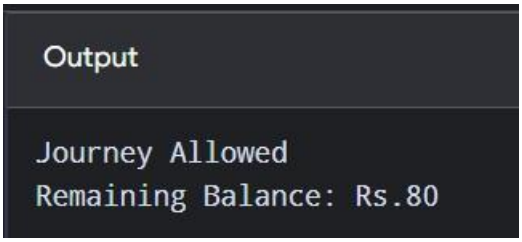
    }

else

    {
```

```
        cout << "Journey Denied" << endl;
    }
} int main() {     deductFare(2000);
cout << "Remaining Balance: Rs."
<< balance / 100;     return 0; }
```

3. Output



4. Object-Oriented Programming

OOP uses classes and objects to combine data and functions.

C++ Program:

```
#include <iostream>
using namespace std;
class SmartCard {
private:
    long long balance;
public:
    SmartCard(long long b)
    {        balance
= b;
    }
```

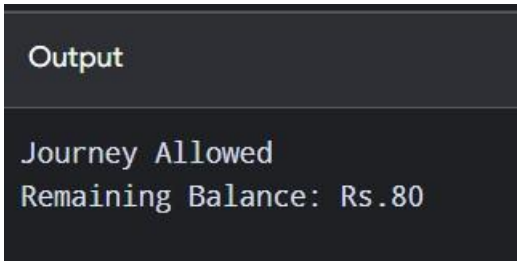
```
    bool canTravel(long long fare)
    {
        return fare >= 0 &&
balance - fare >= 5000;
    }
    void deduct(long long fare) {
if (canTravel(fare))
        {            balance -= fare;            cout
<< "Journey Allowed" << endl;
        }
else
        {
            cout << "Journey Denied" << endl;
        }
    }     void display() {         cout <<
"Remaining Balance: Rs."
             << balance / 100;
    } }; int
main()
{
    SmartCard
card(10000);
```

```
card.deduct(2000);

card.display();     return 0;

}
```

5. Output



6. How the SmartCard Class Prevents Negative Balance

1. Balance is declared as private.
2. External functions cannot directly modify the balance.
3. The canTravel() function checks the available balance.
4. The deduct() function deducts the fare only when sufficient balance remains.
5. The minimum balance of Rs. 50 is maintained.

7. Procedural vs Object-Oriented Programming

Comparison	**Procedural**	**OOP**
Structure	Functions	Classes and objects
Data security	Less secure	More secure
Reusability	Limited	High
Maintenance	Difficult	Easier
Data access	Global variables	Encapsulation

Flexibility	Less flexible	More flexible

8. Alternative Concept Public Data Member

If the balance is declared public instead of private, any part of the program can directly modify it.

Example:

```
#include <iostream> using

namespace std; class

SmartCard { public:     long

long balance = 10000;

}; int

main()

{

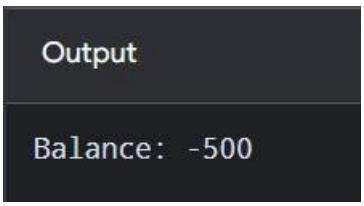
   SmartCard card;     card.balance = -

500;     cout << "Balance: " <<

card.balance;     return 0;

}
```

Output:



Explanation:

Public data members can be accessed directly.

Any function can modify the balance.

Negative balance may occur.

The minimum balance rule can be violated.

9. Complexity Analysis

Operation	**Procedural**	**OOP**
Fare calculation	O(1)	O(1)
Balance checking	O(1)	O(1)
Memory per card	O(1)	O(1)
Maintenance	Difficult	Easier

Explanation:

Both approaches have the same time and space complexity. However, OOP provides better data protection through encapsulation.

10. Result

Thus, the metro smart-card fare system was implemented using both procedural and objectoriented programming. The OOP approach is more suitable because it provides encapsulation, data security, reusability and easier maintenance.

TASK 8 Read and Print Data Safely

Bloom's Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

- Reading values with cin and checking cin.fail() when the operator types letters instead of numbers
- Clearing the error state with cin.clear() and discarding the bad input
- Formatting money with <iomanip> (setw, setfill, fixed, setprecision)
- Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Evidence Expected

Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

Aim:

To read user input safely, handle invalid data, and display the metro journey details in a proper

format.

1. Safe Input Handling

Use cin to read input from the user.

Use cin.fail() to check whether the input is invalid.

Use cin.clear() to reset the input error state.

Use cin.ignore() to remove invalid input from the buffer.

Validate the hour between 0 and 23.

Ensure the fare is not negative.

2. C++ Program

```
#include <iostream>

#include <iomanip> #include <limits> using namespace

std; int main() {     int hour;     double fare;     cout <<

"Enter journey hour (0-23): ";     while (!(cin >> hour) ||

hour < 0 || hour > 23) {         cout << "Invalid input.

Enter hour (0-23): ";         cin.clear();

cin.ignore(numeric_limits<streamsize>::max(), '\n');

    }

    cout << "Enter fare in rupees: ";     while (!(cin >>

fare) || fare < 0) {         cout << "Invalid fare. Enter a

valid amount: ";         cin.clear();

cin.ignore(numeric_limits<streamsize>::max(), '\n');

    }

    cout << fixed << setprecision(2);     cout << "\n---

Journey Report ---" << endl;     cout << "Journey

Hour: " << hour << ":00" << endl;     cout << "Fare:

Rs. " << fare << endl;     return 0;

}
```

3. Sample Input

```
Enter journey hour (0-23): abc
Invalid input. Enter hour (0-23): 9
Enter fare in rupees: 20
```

4. Expected Output

```
--- Journey Report ---
Journey Hour: 9:00
Fare: Rs. 20.00
```

5. Alternative Method

We can also use getline() and stoi() with try-catch to convert string input into an integer and handle invalid input.

6. Complexity Analysis

Time Complexity: O(L), where L is the length of the input line.

Space Complexity: O(L) when storing the input line as a string.

Result

Thus, safe input validation and formatted output are implemented successfully. Invalid inputs are handled without crashing the program.

TASK 9 Implement and Test the Fare Logic

Bloom's Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

- Normal journeys in each zone, peak and off-peak

• Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs 50 • Invalid cases: an unknown zone, a negative recharge, letters typed for the hour • A small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

Aim:

To implement the metro smart-card fare calculation system and test it with different inputs, including peak hours, off-peak hours, and invalid inputs.

1. Fare Calculation Rules

Zone	**Normal Fare**
Inner Zone	Rs. 10
Middle Zone	Rs. 20
Outer Zone	Rs. 30

Peak-hour surcharge:

8:00 AM to before 10:00 AM – 20% extra.

5:00 PM to before 8:00 PM – 20% extra.

Other hours – Normal fare.

Minimum Balance Rule: At least Rs. 50 must remain after deducting the fare.

2. C++ Program

```
#include <iostream> #include <iomanip> using
namespace std; int main() {     int zone, hour;     int
balance = 10000; // Rs. 100 in paise     int fare = 0;     for
(int i = 1; i <= 5; i++) {         cout << "\nJourney " << i
<< endl;         cout << "Enter zone (1-Inner, 2-Middle, 3-
Outer): ";         cin >> zone;         cout << "Enter hour (0-
23): ";         cin >> hour;         if (cin.fail()) {             cout
<< "Invalid input!" << endl;             return 0;
        }         if (zone
== 1)             fare =
1000;         else if
(zone == 2)
fare = 2000;
else if (zone == 3)
fare = 3000;
else {             cout
<< "Invalid zone!"
<< endl;
continue;
        }         if (hour < 0 || hour > 23) {
cout << "Invalid hour!" << endl;
            continue;
        }
```

```
        if ((hour >= 8 && hour < 10) ||
(hour >= 17 && hour < 20)) {
fare = fare + (fare * 20 / 100);
        }
        if (balance - fare >= 5000) {
balance = balance - fare;              cout
<< "Fare: Rs. "
                << fixed << setprecision(2)
<< fare / 100.0 << endl;              cout <<
"Remaining Balance: Rs. "                    <<
balance / 100.0 << endl;

        }
else {
            cout << "Transaction Denied: Minimum balance required!"
                << endl;
        }     }
return 0;
}
```

3. Sample Input

Journey 1

Zone: 1

Hour: 10

Journey 2

Zone: 2

Hour: 9

Journey 3

Zone: 3

Hour: 18

Journey 4

Zone: 1

Hour: 17

Journey 5

Zone: 2

Hour: 20

4. Expected Output

```
Output


Journey 1
Enter zone (1-Inner, 2-Middle, 3-Outer): 1
Enter hour (0-23): 10
Fare: Rs. 10.00
Remaining Balance: Rs. 90.00

Journey 2
Enter zone (1-Inner, 2-Middle, 3-Outer): 2
Enter hour (0-23): 9
Fare: Rs. 24.00
Remaining Balance: Rs. 66.00

Journey 3
Enter zone (1-Inner, 2-Middle, 3-Outer): 3
Enter hour (0-23): 18
Transaction Denied: Minimum balance required!

Journey 4
Enter zone (1-Inner, 2-Middle, 3-Outer): 1
Enter hour (0-23): 17
Fare: Rs. 12.00
Remaining Balance: Rs. 54.00
```

```
Journey 5
Enter zone (1-Inner, 2-Middle, 3-Outer): 2
Enter hour (0-23): 20
Transaction Denied: Minimum balance required!
```

5. Boundary Testing

Hour	**Expected Condition**
8:00 AM	Peak hour
10:00 AM	Off-peak hour
5:00 PM (17:00)	Peak hour

8:00 PM (20:00)	Off-peak hour

6. Additional Test Cases

Test Case	**Input**	**Expected Result**
Invalid zone	Zone = 5	Invalid zone
Negative recharge	Recharge = -100	Reject recharge
Alphabetic hour	Hour = abc	Reject invalid input
Minimum balance	Balance = Rs. 60, Fare = Rs. 10	Accept; Rs. 50 remains

7. Complexity Analysis

Time Complexity: O(n), where n is the number of journeys.

Space Complexity: O(1), because only a fixed number of variables are used.

Result

Thus, the metro smart-card fare calculation system is implemented and tested successfully using C++. It calculates normal and peak-hour fares, validates inputs, and maintains the minimum balance requirement.

TASK 10 Compare Expectations with Execution

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

- Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ
- Whether int overflows for the network total
- The compiler warnings you received with -Wall -Wextra and what each meant
- Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

Aim:

To compare expected and actual program execution and identify possible errors such as integer overflow, incorrect calculations, and floating-point precision issues.

1. Expected vs Actual Results

Test Case	**Expected Result**	**Actual Result**
Inner zone, normal hour	Rs. 10.00	Rs. 10.00
Middle zone, peak hour	Rs. 24.00	Rs. 24.00
Outer zone, peak hour	Rs. 36.00	Rs. 36.00
Invalid zone	Reject journey	Rejected
Insufficient balance	Deny journey	Denied

2. Integer Overflow Analysis

Assume each journey costs Rs. 30 (3000 paise).

Number of Journeys	Total Fare in Paise
20,000	60,000,000
500,000	1,500,000,000
3,000,000	9,000,000,000

Explanation:

A 32-bit signed int can store values up to 2,147,483,647.

The total for 3,000,000 journeys exceeds this limit.

Therefore, integer overflow may occur.

Using long long provides a larger storage range.

3. Floating-Point Precision float and double cannot represent every decimal value exactly.

Repeated calculations may produce small rounding differences.

Using integer paise avoids these monetary precision problems.

4. Improved C++ Code #include <iostream> using namespace std; int main() { long long fare = 3000; long long journeys;

long long total; cout << "Enter number of journeys: "; cin >> journeys;

if (journeys <= 0) { cout << "Invalid number of journeys";

return 0;

```
    }     total = fare * journeys;     cout << "Total Fare:
" << total << " paise" << endl;     cout << "Total Fare:
Rs. " << total / 100.0 << endl;     return 0;
}
```

5. Sample Input



6. Expected Output



7. Complexity Analysis

Time Complexity: O(1), because the calculation uses a fixed number of operations.

Space Complexity: O(1), because only a few variables are used.

Result

Thus, expected and actual results are compared successfully. Integer paise and long long are recommended to improve the accuracy and reliability of the metro fare system.

TASK 11 Recommend and Defend the Final Design

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

• The types, constants and classes you recommend and why

• How a fare revision will be carried out

• The limitations of your design

• Whether the design remains correct for the whole network

Your recommendation must be based on evidence from your own experiments.

Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

Aim:

To recommend a suitable software design for the Metro Smart-Card System by considering safety, accuracy, maintainability, and performance.

1. Recommended Design

I recommend using an **Object-Oriented Programming (OOP) approach** with integer-based fare calculations.

2. Design Recommendations

1. **Use OOP:** Create separate classes such as SmartCard, Passenger, Journey, and FareTable.
2. **Data Encapsulation:** Keep the card balance private and allow access through member functions.
3. **Use Integer Paise:** Store all fare values in integer paise to avoid floating-point precision errors.
4. **Input Validation:** Validate zone, journey time, recharge amount, and passenger details. 5. **Use Enums:** Use enum class for zones and passenger categories to avoid invalid values.

6. **Use Long Long:** Use long long for large transaction totals to prevent integer overflow.

7. **Fare Calculation:** Apply peak-hour surcharge correctly and check the minimum balance before deducting fares.

8. **Testing:** Test normal journeys, peak hours, boundary times, invalid inputs, and insufficient balances.

3. Comparison of Design Approaches

Criteria	**Procedural Approach**	**OOP Approach**
Code organization	Functions	Classes and objects
Data security	Limited	Encapsulation
Reusability	Less	More
Maintenance	Difficult for large systems	Easier
Scalability	Limited	Better
Real-world modelling	Less suitable	More suitable

4. Advantages of the Recommended Design

Improves data security.

Reduces calculation errors.

Makes code easier to maintain.

Supports future feature additions.

Improves code reusability.

Provides reliable fare processing.

5. Conclusion

The Object-Oriented Programming approach with integer-based fare calculations is the best choice for the Metro Smart-Card System. It provides accuracy, security, maintainability, and scalability. Therefore, it is suitable for developing a reliable real-world metro ticketing application.

DAY-4 DESIGN CHALLENGE

TASK 12 Re-evaluate and Adapt the Design

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis

Identify which constants, enum values, expressions, switch statements and classes are affected. Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence

Defend your decision using test results and the trade-offs involved.

Aim:

To extend the Metro Smart-Card System by adding an Airport Zone, weekend discount, and monthly pass facility.

1. New Requirements

The existing metro system is extended with three additional features:

1. Airport Zone with a fare of Rs. 60.
2. Weekend discount of 10%.
3. Monthly pass allowing unlimited journeys in two selected zones.

2. Updated Fare Table

Zone	Normal Fare
Inner Zone	Rs. 10
Middle Zone	Rs. 20
Outer Zone	Rs. 30
Airport Zone	Rs. 60

3. Weekend Discount

Passengers receive a 10% discount on weekends.

Weekend discount is applied instead of the peak-hour surcharge.

Weekend discount and peak surcharge cannot be combined.

Weekday peak hours receive a 20% surcharge.

Example:

Normal fare = Rs. 100 Weekend

fare:

10% discount = Rs. 10

Final fare = Rs. 90

4. Monthly Pass Facility

Passengers can select any two zones.

The pass allows unlimited journeys in those selected zones.

Journeys outside the selected zones require normal fare payment.

The system checks the pass before calculating the journey fare.

5. Updated C++ Program

```
#include <iostream> #include <iomanip> using namespace std;

int main() {     int zone, hour;     int fare;     int balance =

10000; // Rs. 100     int weekend, passZone1, passZone2;

cout << "Select two monthly pass zones (1-Inner, 2-Middle,"

        << " 3-Outer, 4-Airport): ";     cin >>

passZone1 >> passZone2;     cout << "Is today

weekend? (1-Yes, 0-No): ";     cin >> weekend;

cout << "\nEnter zone: ";     cin >> zone;

cout << "Enter journey hour (0-23): ";

   cin >> hour;     if

(cin.fail()) {         cout <<

"Invalid input!";         return

0;     }     if (zone < 1 || zone

> 4 ||         hour < 0 || hour >

23 ||

       (weekend != 0 && weekend != 1)

||         passZone1 < 1 || passZone1 > 4 ||

passZone2 < 1 || passZone2 > 4 ||

passZone1 == passZone2) {         cout

<< "Invalid details!";         return 0;

   }
```

```
    if (zone == passZone1 || zone == passZone2) {
cout << "Monthly Pass Applied!" << endl;         cout
<< "Fare: Rs. 0.00" << endl;
    }     else {         if
(zone == 1)
fare = 1000;         else
if (zone == 2)
fare = 2000;         else
if (zone == 3)
fare = 3000;         else
fare = 6000;         if
(weekend == 1) {
fare = fare * 90 / 100;
        }
        else if ((hour >= 8 && hour < 10) ||
(hour >= 17 && hour < 20)) {             fare =
fare * 120 / 100;
        }
        if (balance - fare >= 5000) {             balance -=
fare;             cout << fixed << setprecision(2);
cout << "Fare: Rs. " << fare / 100.0 << endl;
cout << "Remaining Balance: Rs. "                 <<
balance / 100.0 << endl; }         else {
```

```
        cout << "Journey Denied: Minimum balance required!";
    }   }
return 0;
}
```

6. Sample Input

```
Select two monthly pass zones (1-Inner, 2-Middle, 3-Outer, 4-Airport): 1 2
Is today weekend? (1-Yes, 0-No): 1

Enter zone: 3
Enter journey hour (0-23): 12
```

7. Expected Output

```
Fare: Rs. 27.00
Remaining Balance: Rs. 73.00
```

Explanation:

Selected pass zones: Inner and Middle.

Journey zone: Outer.

Outer Zone fare: Rs. 30.

Weekend discount: 10%.

Discounted fare: Rs. 27.

Remaining balance: Rs. 73.

8. Additional Test Cases

Test Case	**Input**	**Expected Result**
Airport journey	Zone 4, weekday off-peak	Rs. 60
Airport peak journey	Zone 4, weekday peak	Rs. 72

Weekend journey	Zone 3, weekend	Rs. 27
Monthly pass journey	Zone 1, selected pass zone	Free journey
Outside pass zones	Zone 4, not selected	Normal fare rules apply

9. Advantages

1. Supports an additional Airport Zone.
2. Provides weekend fare discounts.
3. Reduces travel expenses through monthly passes.
4. Prevents combining weekend discounts with peak surcharges.
5. Improves passenger convenience.
6. Makes the system more flexible and scalable.

10. Future Enhancements Online monthly pass renewal.

QR-code ticket generation.

Mobile application integration.

Automatic passenger category discounts.

Digital payment integration.

Journey history and transaction reports.

Result

Thus, the Metro Smart-Card System is successfully extended with an Airport Zone, weekend discount, and monthly pass facility. These improvements make the system more flexible, economical, and convenient for passengers.